TP 1 : Utilisation d'OpenCV

L'objectif de ce TP est d'utiliser la bibliothèque d'*OpenCV*[1] *(Open Source Computer Vision Library)*. OpenCV est une bibliothèque graphique et open-source. Elle dispose de nombreuse de fonctionnalités permettant de créer les interface graphique ainsi que des outils dans le domaine de traitement d'images et de vidéo.

Dans ce TP, nous allons utiliser l'OpenCV 4.x. Vous trouverez une documentation complète sur `https://opencv.org/releases/` et `https://docs.opencv.org/4.12.0/`.

Les fichiers de supports du TP sont à récupérer sur Arche.

Dépôt sur Arche : Vous rendez le fichier `main.cpp` et un petit rapport en pdf contenant les résultats d'exécution de votre programme.

Exercice 1 — Un premier programme avec OpenCV

Nous commençons par un programme simple en OpenCV qui charge une image et l'afficher dans un fenêtre. Pour cela, nous allons utiliser avec les fonctions suivantes :

```
//Créer une fênêtre
void namedWindow ( const String & winname, int flags = WINDOW_AUTOSIZE ) ;
//Charger une image
Mat imread ( const String & filename, int flags = IMREAD_COLOR ) ;
//Sauvegarder une image
Mat bool imwrite(const string& filename, InputArray img, ... ) ;
//Afficher une image dans une fenêtre
void imshow ( const String & winname, InputArray mat ) ;
//Attendre l'appui d'une touche
int waitKey(int delay=0);
```

Renseignez les fonctions et leurs paramètres sur la documentation de la bibliothèques OpenCV sur `https://docs.opencv.org/4.12.0/d7/dfc/group__highgui.html`.

Un exemple de programme d'affichage d'une image est donné dans le fichier `main.cpp`



1. `https://opencv.org`

```
#include ...

string window_name = "OpenCv Demo";

int main( int argc, char** argv )
{
    string imageName("images/daisy.png");
    if( argc > 1)
        imageName = argv[1];
    Mat image;
    image = imread(imageName.c_str(), IMREAD_COLOR);
    if( image.empty() )
    {
        cout <<  "Erreur de chargment d'image" << endl ;
        return -1;
    }
    namedWindow( window_name, WINDOW_AUTOSIZE );
    imshow( window_name, image );
    waitKey(0);
    return 0;
}
```

Allez sur Arche et téléchargez les supports de ce TP, puis compilez le programme avec le `Makefile` ou le `CMakeLists.txt`, et lancez le programme compilé.

```
#Pour compiler avec Makefile
make

#Pour compiler avec CMakeLists.txt
cmake .
make

#Pour executer le programme compilé
./main
./main images/daisy.png
./main images/lotus.png
```

Exercice 2 — Convertir une image

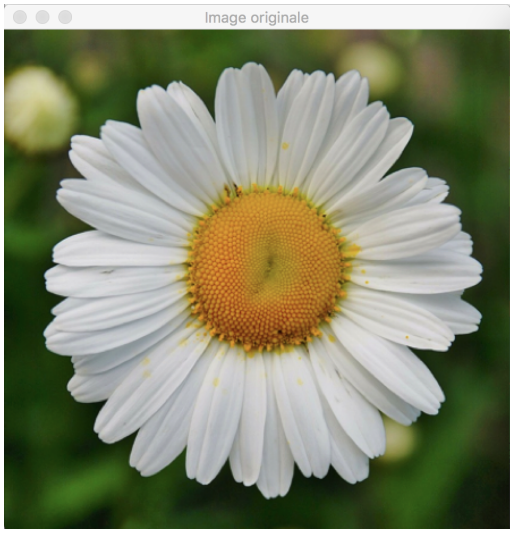
Dans cette exercice, nous allons convertir une image couleur en niveau de gris et l'afficher sur une fenêtre. Pour cela, nous avons besoin la fonction :

```
void cvtColor(InputArray src, OutputArray dst, int code, int dstCn=0 ) ;
```

Modifiez votre programme `main.cpp` pour :

— changer le nom de la fenêtre de "OpenCv Demo" en "Image originale"
— créer une deuxième fenêtre avec le nom de "Image convertie"
— Convertir l'image couleur en niveaux de gris en utilisant la fonction `cvtColor` avec `code=COLOR_BGR2GRAY`
— Afficher l'image en niveaux de gris sur la fenêtre "Image convertie"

Vous devez obtenir le résultat suivant :





Nous allons maintenant extraire les trois canaux BGR (blue-green-red) de l'image en couleur et afficher sur la deuxième fenêtre selon la touche appuyée : b, g ou r. Pour cela, nous allons créer trois images, un pour chaque canal, puis parcourir l'image et récupérer les couleurs de pixels associées au canal à extraire.

Pour extraire des canaux, nous avons besoin les fonctions suivantes [2] :

```
//créer une image avec Mat constructor (dont le type d'image est CV_8UC3)
Mat::Mat(Size size, int type, const Scalar& s) ;
//récupérer la taille d'une image
Size s = img.size() ;
//récupérer l'hauteur et largeur d'une image
int largeur = img.cols ; ou int largeur = s.width ;
int hauteur = img.rows ; ou int hauteur = s.height;
//récupérer la couleur BGR d'un pixel (image CV_8UC3)
Vec3b intensity = img.at<Vec3b>(y, x);
uchar bleu = intensity.val[0];
uchar vert = intensity.val[1];
uchar rouge = intensity.val[2];
//récupérer le niveau gris d'un pixel (image CV_8U ou CV_8UC1)
uchar intensity = img.at<uchar>(y, x);
```

2. https://docs.opencv.org/2.4/doc/tutorials/core/mat_the_basic_image_container/mat_the_basic_image_container.html

Pour les événements des touches du clavier, nous avons besoin la fonction suivante :

```
//attendre l'appuie d'une touche et retourne la touche appuyée
int c= waitKey(20);
//vérifier la touche appuyée
if (c=='r') ...
```

Modifiez votre programme `main.cpp` pour
- créer 3 images pour les trois canaux BGR en utilisant le constructeur de Mat avec `size=image.size(), type=CV_8UC3` et `s=Scalar(0,0,0)`.
- parcourir l'image origine et récupérer les canaux BGR pour stocker dans chaque image créée précédente
- dans une boucle `while(true)`
  - détecter l'appuie sur une touche avec `waitKey(0)`
  - afficher, en fonction de la touche appuyée 'b','g', 'r' ou 'y', l'image de canal correspondant bleu, vert, rouge ou image en niveaux gris sur la deuxième fenêtre. Pour n'importe quelle autres touches, quitter le programme.

Exécuter votre programme avec différentes images de test.

Exercice 3 — Changement d'espace de couleur

Nous allons changer d'espace de couleurs d'une image RGB en
- image CMYB

$$\begin{cases} R' &= \frac{R}{255} \\ G' &= \frac{G}{255} \\ B' &= \frac{B}{255} \end{cases} \qquad \begin{cases} K &= 1 - max(R', G', B') \\ R'' &= \frac{1-R'-K}{1-K} \\ G'' &= \frac{1-G'-K}{1-K} \\ B'' &= \frac{1-B'-K}{1-K} \end{cases}$$

- image YCbCr

$$\begin{cases} R' &= \frac{R}{255} \\ G' &= \frac{G}{255} \\ B' &= \frac{B}{255} \end{cases} \qquad \begin{cases} B'' = Y = K_r R' + K_g G' + K_b B' \\ G'' = Cb = \frac{W(B'-Y)}{(1-K_b)} \\ R'' = Cr = \frac{W(R'-Y)}{(1-K_r)} \end{cases}$$

avec $K_r = 0.299, K_b = 0.114, K_g = 0.587$ et $W = 0.5$.
- image par une multilication matricielle avec la matrice suivante

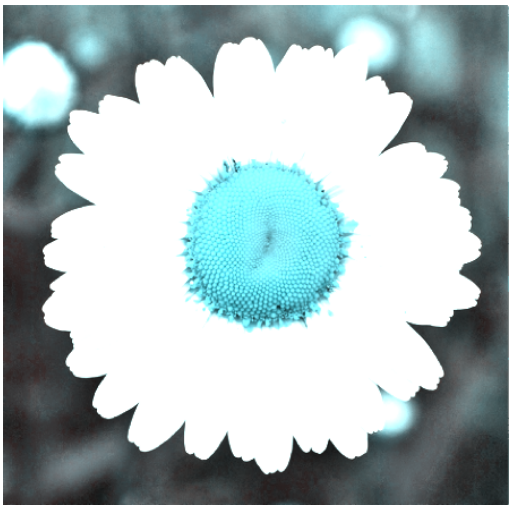
$$\frac{1}{255}\begin{pmatrix} 0.6 & 0.8 & 0.9 \\ 0.7 & 0.8 & 0.8 \\ 0.9 & 0 & 0 \end{pmatrix}$$

Pour cela, nous allons utiliser les fonctions suivantes :

```
//créer une image avec le type d'image est CV_32FC3
Mat::Mat(Size size, int type, const Scalar& s) ;
//modifier la couleur BGR d'un pixel (image CV_32FC3)
img.at<Vec3f>(y, x).val[0]=b;
img.at<Vec3f>(y, x).val[1]=g;
img.at<Vec3f>(y, x).val[2]=r;
```

Modifiez votre programme `main.cpp` pour réaliser ce changement d'espace de couleur. Vous devez obtenir les résultats suivants :



Exercice 4 — Sélection par couleur

Nous allons créer une petit interface permettant de sélectionner tous les pixels d'une couleur choisie sur l'image. Plus précisément, l'utilisateur est invité de cliquer sur l'image la couleur qu'il veut sélectionner. Nous afficherons l'ensemble des pixels ayant la même couleur. Pour cela, nous avons besoin de définir le handler [3] de souris pour la fenêtre spécifiée. Il s'agit de la fonction :

```
void setMouseCallback(const string& winname, MouseCallback onMouse, ...) ;
```

et le `MouseCallback` pour le traitement des événements de la souris est prototypé comme :

```
void onMouse(int event, int x, int y, int , void* ) ;
```

dans lequel :

3. https://docs.opencv.org/2.4/modules/highgui/doc/user_interface.html?highlight=setmousecallback

— `winname` est le nom de la fenêtre sur laquelle on associe les événements de la souris
— `onMouse` est le nom de la fonction handler du callback des événements de la souris
— `event` est l'événement associé au bouton de la souris [4], eg. `EVENT_LBUTTONDOWN`, `EVENT_RBUTTONDOWN`, ...
— `x,y` sont les coordonnées de la souris sur la fenêtre.

Modifiez votre programme `main.cpp` pour
— associer le handler de souris pour la fenêtre "Image originale"
— définir le callback pour le bouton gauche de la souris afin de récupérer la couleur du pixel à position où l'utilisateur clique sur l'image
— dans le callback, créer une image contenant les images de même couleur du pixel sélectionné et afficher sur la fenêtre "Image convertie" avec le prototype suivant :

```
void selectionerCouleur(int event, int x, int y, int , void* ) ;
```

Pour écrire une image, vous pouvez utiliser la fonction :

```
bool imwrite(const string& filename, InputArray img, ...) ;
```

Nous allons maintenant relaxer une peu la condition de couleur. Au lieu de sélectionner les pixels de même couleur, nous sélectionnerons les pixels dont la couleur est dans une intervalle autour de la couleur sélectionnée. Plus précisément, soit $I(x, y)$ est la couleur du pixel sélectionné par l'utilisateur, nous allons prendre les pixels $I(x', y')$ tel que sa couleur est $|I(x, y) - I(x', y')| \leqslant seuil$ sur le trois canaux BGR. Le seuil est contrôlé grâce au trackbar [5] d'OpenCV. Pour cela, nous avons besoin de créer un trackbar associé à la fenêtre spécifiée avec la fonction :

```
int createTrackbar(const string& trackbarname, const string& winname,
              int* value, int max_value, TrackbarCallback onChange=0, ...)
;
```

et le `TrackbarCallback` pour le traitement d'événement du trackbar est prototypé comme :

```
void onChange( int, void* ) ;
```

dans lequel :
— `trackbarname` est le nom du trackbar
— `winname` est le nom de la fenêtre sur laquelle on associe le trackbar
— `value` stocke la valeur actuelle du trackbar
— `max_value` est la valeur maximales du trackbar
— `onChange` est le nom de la fonction handler du callback d'événement du trackbar.

Modifiez votre programme `main.cpp` pour
— associer un trackbar pour la fenêtre "Image originale"
— définir le callback du trackbar pour le seuil avec le prototype suivant

```
void selectionnerSeuil( int, void* ) ;
```

— dans ce callback, modifier l'image de sélection par couleur selon la condition de seuil et afficher sur la fenêtre "Image convertie"

4. https://docs.opencv.org/4.12.0/d7/dfc/group__highgui.html
5. https://docs.opencv.org/2.4/modules/highgui/doc/user_interface.html?highlight=setmousecallback#createtrackbar

— appeler la fonction `selectionnerSeuil (0,0)` juste après `createTrackbar` pour initialiser
— ajouter dans l'ensemble de touche appuyée, si l'utilisateur appuie sur la touche 's', on sauvegarde l'image résultant avec le nom "resultat.png".

Exécuter votre programme avec différentes images de test. Vous devez obtenir le résultat suivant sur l'image `daisy.png`.



